PS 1

```
mysql> create database Banking;
Query OK, 1 row affected (0.02 sec)

mysql> use Banking;
Database changed
mysql> create table Customer(
    -> cust_id INT PRIMARY KEY,
    -> cust_name VARCHAR(50),
    -> city VARCHAR(30),
    -> phone VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql> CREATE TABLE Branch (
    ->     branch_id INT PRIMARY KEY,
    ->     branch_name VARCHAR(30),
    ->     branch_city VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> create table Account (
    -> acc_no INT PRIMARY KEY,
    -> cust_id INT,
    -> branch_id INT,
    -> balance DECIMAL(10,2),
    -> FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    -> FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql> CREATE TABLE Loan (
    ->     loan_no INT PRIMARY KEY,
    ->     cust_id INT,
    ->     branch_id INT,
    ->     amount DECIMAL(10,2),
    ->     FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    ->     FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

1. Display all customers in alphabetical order of their names.

```
select * from Customer ORDER BY cust_name ASC;
+---------+-----------+--------+------------+
| cust_id | cust_name | city   | phone      |
+---------+-----------+--------+------------+
|       8 | Akash     | Nashik | 9876543217 |
```

```
|       1 | Amit      | Pune   | 9876543210 |
|       6 | Karan     | Delhi  | 9876543215 |
|       5 | Neha      | Mumbai | NULL       |
|       4 | Pooja     | Pune   | 9876543213 |
|       2 | Priya     | Mumbai | 9876543211 |
|       3 | Rahul     | Nashik | NULL       |
|       7 | Riya      | Pune   | 9876543216 |
|       9 | Sneha     | Mumbai | 9876543218 |
|      10 | Varun     | Delhi  | NULL       |
+---------+-----------+--------+------------+
10 rows in set (0.00 sec)
```

2. Display the names and cities of customers whose names start with the letter 'P'.

```
 select cust_name, city from Customer WHERE cust_name LIKE 'P%';
```

3. Display customers who live in Pune or Mumbai and whose phone number is not NULL.

```
select * FROM Customer WHERE city = 'Pune' || city = 'Mumbai' AND phone IS NOT NULL;
```

4. Display all customers whose phone number is NULL.

```
select * from Customer WHERE phone is NULL;
```

5. Display the average account balance for each branch.

```
select b.branch_name, AVG(a.balance) AS avg_balance
    -> FROM Branch b
    -> JOIN Account a ON b.branch_id = a.branch_id
    -> GROUP BY b.branch_name;
```

6. Display the branch-wise average account balance and show only those branches where the average balance is greater than ₹40,000.

```
select b.branch_name, AVG(a.balance) AS avg_balance
    -> FROM Branch b
    -> JOIN Account a ON b.branch_id = a.branch_id
    -> GROUP BY b.branch_name
-> HAVING AVG(a.balance) > 40000;
```

7. Display each account number along with its balance after adding 10% interest. Do not modify the actual balance.

```
select acc_no, balance, balance * 1.10 AS update_balance from Account;
```

8. Update the phone number of a specific customer using the customer's name.

```
UPDATE Customer SET Phone = '999999999' WHERE cust_name = 'Amit';
```

9. Delete all customer records for customers who live in Pune.

```
mysql> DELETE FROM Account WHERE cust_id IN (
    -> SELECT cust_id FROM Customer WHERE city = 'Pune'
    -> );
Query OK, 3 rows affected (0.03 sec)

mysql> DELETE FROM Loan WHERE cust_id IN (
```

```
    -> SELECT cust_id FROM Customer WHERE city = 'Pune'
    -> );
Query OK, 3 rows affected (0.02 sec)

mysql> DELETE FROM Customer WHERE city = 'Pune';
Query OK, 3 rows affected (0.02 sec)
```

10. Display customers who either have an account or have a loan using UNION.

```
mysql> select c.cust_id, c.cust_name
    -> FROM Customer c
    -> JOIN Account a ON c.cust_id = a.cust_id
    -> UNION
    -> select c.cust_id, c.cust_name
    -> FROM Customer c
-> JOIN Loan l ON c.cust_id = l.cust_id;
```

PS 2

1. Display student names along with their course names and marks.

```
select s.name, e.course, e.marks FROM Student s
   -> INNER JOIN Enrollments e
   -> ON s.student_id = e.student_id;
```

2. Display all students along with their course names and marks, including students who are not enrolled in any course.

```
mysql> select s.name, e.course, e.marks
   → FROM Student s
   -> LEFT JOIN Enrollments e
   -> ON s.student_id = e.student_id;
```

3. Display all enrollments along with the corresponding student names.

```
mysql> select e.enroll_id, e.student_id, e.course, e.marks, s.name
   -> FROM Enrollments e
   -> LEFT JOIN Student s
   -> ON e.student_id = s.student_id;
```

4. Display all possible combinations of students and courses. //isme table me har name ke sath possible course aayeing axb ke form mien (10x10=100)

```
mysql> select s.name, e.course FROM Student s
   -> CROSS JOIN Enrollments e;
```

5. Display students whose marks are greater than the average marks of all enrollments .

```
mysql> select s.name, e.marks FROM Student s
   -> JOIN Enrollments e
   -> ON s.student_id = e.student_id
   -> WHERE e.marks > (
   -> SELECT AVG(marks) FROM Enrollments
   -> );
```

6. Display students who have obtained marks greater than 80

```
mysql> select s.name, e.marks FROM Student s
   -> JOIN Enrollments e
   -> ON s.student_id = e.student_id
   -> WHERE e.marks > 80;
```

7. Display students who are enrolled in the DBMS course

```
mysql> select s.name, e.course
   -> FROM Student s
   -> JOIN Enrollments e
   -> ON s.student_id = e.student_id
   -> WHERE e.course = 'DBMS';
```

8. Display the student ID(s) who have obtained the highest marks

```
mysql> select student_id
   -> FROM Enrollments
```

```
   -> WHERE marks = (
   -> SELECT MAX(marks) FROM Enrollments
   -> );
```

9. Create a view named student_course_view displaying student names, course names, and marks. Display all records from the view and update the marks of a student through the view.

```
mysql> create view student_course_view AS
   -> select s.student_id, s.name, e.course, e.marks
   -> FROM Student s
   -> JOIN Enrollments e
   -> ON s.student_id = e.student_id;
mysql> UPDATE student_course_view
   -> SET marks = 90
   -> WHERE student_id = 1 AND course = 'DBMS';
```

10. Create an index on the name column of the Students table and a unique index on the enroll_id column of the Enrollments table

```
mysql> create index idx_student_name ON Student(name);
mysql> create index idx_enroll_id ON Enrollments(enroll_id);
mysql> show index from Student;
```

PS 3

1. Display student names and the courses in which they are enrolled.

```
mysql> select s.name, e.course
    -> FROM Student s
    -> INNER JOIN Enrollments e
    -> ON s.student_id = e.student_id;
```

2. Display all students and their course details, including students who are not enrolled in any course.

```
mysql> select s.name, e.course, e.marks
    -> FROM Student s
    -> LEFT JOIN Enrollments e
    -> ON s.student_id = e.student_id;
```

3. Display the names of students enrolled in DBMS along with their marks.

```
mysql> select s.name, e.marks
    -> FROM Student s
    -> JOIN Enrollments e
    -> ON s.student_id = e.student_id
    -> ORDER BY e.course = 'DBMS';
```

4. Display enrollments that do not have a matching student record.

```
mysql> select e.enroll_id, e.student_id, e.course, e.marks
    -> FROM Enrollments e
    -> LEFT JOIN Student s
    -> ON e.student_id = s.student_id
    -> WHERE s.student_id IS NULL;
```

5. Display students who have obtained marks less than the average marks of all enrollments.

```
mysql> SELECT s.name, e.marks
    -> FROM Student s
    -> JOIN Enrollments e
    -> ON s.student_id = e.student_id
    -> WHERE e.marks < (
    ->     SELECT AVG(marks)
    ->     FROM Enrollments
    -> );
```

6. Display students who have obtained marks equal to the highest marks.

```
mysql> select s.name, e.marks
    -> FROM Student s
    -> JOIN Enrollments e
    -> ON s.student_id = e.student_id
    -> WHERE e.marks = (
    -> SELECT MAX(marks) FROM Enrollments
    -> );
```

7. Display students who belong to Pune.

```
mysql> select * From Student WHERE city = 'Pune';
```

8. Display students enrolled in Python.

```
mysql> select s.name, e.course
    -> FROM Student s
    -> JOIN Enrollments e
    -> ON s.student_id = e.student_id
    -> WHERE e.course = 'Python';
```

9. Create a view named course_average_view displaying course names and their average marks. Display all records from the view.

```
mysql> create view course_average_view AS
    -> SELECT course, AVG(marks) AS avg_marks
    -> FROM Enrollments
-> GROUP BY course;

mysql> select * from course_average_view;
```

(to drop use drop view course_average_view)

10. Create an index on Students.city and a unique index on Students.student_id.

```
mysql> create INDEX idx_students_city ON Student(city);
mysql> create UNIQUE INDEX idx_student_id ON Student(student_id);

mysql> show index from Student;
```

PS 4

```
CREATE TABLE Stud_Marks (
   rollno NUMBER(3) PRIMARY KEY,
   name VARCHAR2(30),
   total_marks NUMBER(4)
);

CREATE TABLE Result (
   Roll NUMBER(3) PRIMARY KEY,
   Name VARCHAR2(30),
   Class VARCHAR2(30)
);

INSERT INTO Stud_Marks VALUES (1, 'Amit', 1210);
INSERT INTO Stud_Marks VALUES (2, 'Neha', 1050);
INSERT INTO Stud_Marks VALUES (3, 'Rahul', 995);
INSERT INTO Stud_Marks VALUES (4, 'Sneha', 960);
INSERT INTO Stud_Marks VALUES (5, 'Rohan', 925);
INSERT INTO Stud_Marks VALUES (6, 'Pooja', 900);
INSERT INTO Stud_Marks VALUES (7, 'Vikas', 890);
INSERT INTO Stud_Marks VALUES (8, 'Anjali', 850);
INSERT INTO Stud_Marks VALUES (9, 'Kiran', 825);
INSERT INTO Stud_Marks VALUES (10, 'Manoj', 790);

COMMIT;

CREATE OR REPLACE PROCEDURE proc_Grade IS
BEGIN
   FOR s IN (
      SELECT rollno, name, total_marks
      FROM Stud_Marks
   )
   LOOP
      IF s.total_marks >= 990 AND s.total_marks <= 1500 THEN
         INSERT INTO Result
         VALUES (s.rollno, s.name, 'Distinction');

      ELSIF s.total_marks >= 900 THEN
         INSERT INTO Result
         VALUES (s.rollno, s.name, 'First Class');

      ELSIF s.total_marks >= 825 THEN
         INSERT INTO Result
         VALUES (s.rollno, s.name, 'Higher Second Class');

      ELSE
         INSERT INTO Result
         VALUES (s.rollno, s.name, 'Fail');
```

```
        END IF;
    END LOOP;

    COMMIT;
END;
/


EXEC proc_Grade;

SELECT * FROM Result;
```

PS 5

```
SQL> create table area(radius NUMBER, area NUMBER);

SQL> CREATE OR REPLACE FUNCTION
  2  CalculateArea(pRadius NUMBER)
  3  RETURN NUMBER
  4  IS
  5     VArea NUMBER;
  6  BEGIN
  7     VArea := 3.14*pRadius*pRadius;
  8     RETURN VArea;
  9  END;
 10  /

SQL> BEGIN
  2  FOR r IN 5..9
  3  LOOP
  4     INSERT INTO area(radius,area) VALUES (r,CalculateArea(r));
  5  END LOOP;
  6  COMMIT;
  7  END;
  8  /

SQL> select * from area;
```

PS 6

```
SQL> create table Employee(
  2  Emp_ID NUMBER(5) PRIMARY KEY,
  3  Emp_name VARCHAR2(30) NOT NULL,
  4  Department VARCHAR2(20),
  5  Salary NUMBER(10,2)
  6  );
SQL> INSERT INTO Employee VALUES (101, 'Amit Sharma', 'IT', 45000);
COMMIT;

SET SERVEROUTPUT ON;

DECLARE
   CURSOR emp_cur IS
      SELECT Emp_ID, Emp_Name, Department, Salary
      FROM Employee
      WHERE Salary > 30000;

   v_id   Employee.Emp_ID%TYPE;
   v_name Employee.Emp_Name%TYPE;
   v_dept Employee.Department%TYPE;
   v_sal  Employee.Salary%TYPE;

BEGIN
   DBMS_OUTPUT.PUT_LINE('Employees with Salary greater than Rs. 30000');
   DBMS_OUTPUT.PUT_LINE('-----------------------------------------');

   OPEN emp_cur;

   LOOP
      FETCH emp_cur INTO v_id, v_name, v_dept, v_sal;

      EXIT WHEN emp_cur%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         v_id || ' ' || v_name || ' ' ||
         v_dept || ' ' || v_sal
      );
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('-----------------------------------------');
   DBMS_OUTPUT.PUT_LINE('Total records fetched: ' || emp_cur%ROWCOUNT);

   CLOSE emp_cur;
END;
/
```

PS 7

```
CREATE TABLE Library (
   book_id NUMBER(5) PRIMARY KEY,
   book_name VARCHAR2(50),
   author VARCHAR2(50),
   category VARCHAR2(30),
   date DATE,
   operation VARCHAR2(10)
);

CREATE TABLE Library_Audit (
   book_id NUMBER(5),
   book_name VARCHAR2(50),
   author VARCHAR2(50),
   category VARCHAR2(30),
   date DATE,
   operation VARCHAR2(10)
);

INSERT INTO Library VALUES (1, 'Wings of Fire', 'A.P.J. Abdul Kalam', 'Biography', DATE '2024-
01-10', 'INSERT');
INSERT INTO Library VALUES (2, 'The Alchemist', 'Paulo Coelho', 'Fiction', DATE '2024-02-12',
'INSERT');
INSERT INTO Library VALUES (3, '1984', 'George Orwell', 'Fiction', DATE '2024-03-15', 'INSERT');
INSERT INTO Library VALUES (4, 'Discovery of India', 'Jawaharlal Nehru', 'History', DATE '2024-
04-20', 'INSERT');
INSERT INTO Library VALUES (5, 'Ignited Minds', 'A.P.J. Abdul Kalam', 'Motivation', DATE '2024-
05-05', 'INSERT');
INSERT INTO Library VALUES (6, 'Godan', 'Munshi Premchand', 'Novel', DATE '2024-06-18',
'INSERT');
INSERT INTO Library VALUES (7, 'Hamlet', 'William Shakespeare', 'Drama', DATE '2024-07-22',
'INSERT');
INSERT INTO Library VALUES (8, 'The Guide', 'R.K. Narayan', 'Novel', DATE '2024-08-14',
'INSERT');
INSERT INTO Library VALUES (9, 'Malgudi Days', 'R.K. Narayan', 'Short Stories', DATE '2024-09-
09', 'INSERT');
INSERT INTO Library VALUES (10, 'India 2020', 'A.P.J. Abdul Kalam', 'Technology', DATE '2024-
10-01', 'INSERT');

COMMIT;

CREATE OR REPLACE TRIGGER Library_Audit_Trigger
BEFORE UPDATE OR DELETE ON Library
FOR EACH ROW
BEGIN
   IF UPDATING THEN
      INSERT INTO Library_Audit
      VALUES (
```

```
            :OLD.book_id,
            :OLD.book_name,
            :OLD.author,
            :OLD.category,
            :OLD.date,
            'UPDATE'
        );

    ELSIF DELETING THEN
        INSERT INTO Library_Audit
        VALUES (
            :OLD.book_id,
            :OLD.book_name,
            :OLD.author,
            :OLD.category,
            :OLD.date,
            'DELETE'
        );
    END IF;
END;
/

UPDATE Library
SET book_name = 'Advanced C Programming'
WHERE book_id = 1;

COMMIT;

SELECT * FROM Library WHERE book_id = 1;
SELECT * FROM Library_Audit;


DELETE FROM Library
WHERE book_id = 2;

COMMIT;

SELECT * FROM Library_Audit;
```

PS 8

```
mysql> create database temp;
mysql> use temp;
mysql> create table Student(
    -> roll INT PRIMARY KEY,
    -> name VARCHAR(50),
    -> marks INT
    -> );
mysql> create user 'temp'@'localhost' IDENTIFIED BY 'temp';
mysql> grant all privileges ON temp.* TO 'temp'@'localhost';


 nano python9.py
import pymysql

# Connect to MySQL
mydb = pymysql.connect(
    host="localhost",
    user="temp",
    password="temp",
    database="temp"
)

mycursor = mydb.cursor()

print("1. Insert")
print("2. Update")
print("3. Delete")

choice = input("Enter choice (1/2/3): ")

if choice == "1":
    roll = int(input("Enter roll no: "))
    name = input("Enter name: ")
    marks = int(input("Enter marks: "))

    sql = "INSERT INTO Student (roll, name, marks) VALUES (%s, %s, %s)"
    val = (roll, name, marks)

    mycursor.execute(sql, val)
    mydb.commit()
    print(mycursor.rowcount, "record inserted.")

elif choice == "2":
    name = input("Enter student name to update: ")
    marks = int(input("Enter new marks: "))

    sql = "UPDATE Student SET marks = %s WHERE name = %s"
```

```
    val = (marks, name)

    mycursor.execute(sql, val)
    mydb.commit()
    print(mycursor.rowcount, "record updated.")

elif choice == "3":
    name = input("Enter student name to delete: ")

    sql = "DELETE FROM Student WHERE name = %s"
    val = (name,)

    mycursor.execute(sql, val)
    mydb.commit()
    print(mycursor.rowcount, "record deleted.")

else:
    print("Invalid choice.")

mycursor.close()
mydb.close()


 python3 python9.py
mysql> use temp;
mysql> select * FROM Student;
```

PS 9
sudo systemctl start mongod
nano a.py
python3 a.py

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")

db = client["college"]
student = db["student"]

print("1. Insert")
print("2. Update")
print("3. Delete")

choice = input("Enter choice: ")

if choice == "1":
    roll_no = input("Enter roll no: ")
    name = input("Enter name: ")
    marks = input("Enter marks: ")

    student.insert_one({
        "roll_no": roll_no,
        "name": name,
        "marks": marks
    })
```

```
    print("Record inserted")

elif choice == "2":
    name = input("Enter name: ")
    marks = input("Enter new marks: ")

    student.update_one(
        {"name": name},
        {"$set": {"marks": marks}}
    )

    print("Record updated")

elif choice == "3":
    name = input("Enter name: ")
    student.delete_one({"name": name})

    print("Record deleted")

else:
    print("Invalid choice")

client.close()
```

PS 10

```
CREATE TABLE Branch (
   branch_name VARCHAR(50) PRIMARY KEY,
   branch_city VARCHAR(50) NOT NULL,
   assets DECIMAL(15,2) NOT NULL CHECK (assets >= 0)
);

CREATE TABLE Account (
   acc_no INT PRIMARY KEY,
   branch_name VARCHAR(50) NOT NULL,
   balance DECIMAL(12,2) NOT NULL CHECK (balance >= 0),
   FOREIGN KEY (branch_name)
      REFERENCES Branch(branch_name)
);


CREATE TABLE Customer (
   cust_name VARCHAR(50) PRIMARY KEY,
   cust_street VARCHAR(100) NOT NULL,
   cust_city VARCHAR(50) NOT NULL
);

CREATE TABLE Depositor (
   cust_name VARCHAR(50),
   acc_no INT,
   PRIMARY KEY (cust_name, acc_no),
   FOREIGN KEY (cust_name)
      REFERENCES Customer(cust_name),
   FOREIGN KEY (acc_no)
      REFERENCES Account(acc_no)
);
CREATE TABLE Loan (
   loan_no INT PRIMARY KEY,
   branch_name VARCHAR(50) NOT NULL,
   amount DECIMAL(12,2) NOT NULL CHECK (amount >= 0),
   FOREIGN KEY (branch_name)
      REFERENCES Branch(branch_name)
);
CREATE TABLE Borrower (
   cust_name VARCHAR(50),
   loan_no INT,
   PRIMARY KEY (cust_name, loan_no),
   FOREIGN KEY (cust_name)
      REFERENCES Customer(cust_name),
   FOREIGN KEY (loan_no)
      REFERENCES Loan(loan_no)
```

```
);

INSERT INTO Branch VALUES
('Akurdi', 'Pune', 9000000),
('Chinchwad', 'Pune', 12000000);

INSERT INTO Customer VALUES
('Amit', 'MG Road', 'Pune'),
('Temp', 'FC Road', 'Pune');

INSERT INTO Account VALUES
(101, 'Chinchwad', 40000),
(102, 'Chinchwad', 60000);

INSERT INTO Depositor VALUES
('Amit', 101),
('Temp', 102);

INSERT INTO Loan VALUES
(201, 'Akurdi', 600000),
(202, 'Akurdi', 400000);

INSERT INTO Borrower VALUES
('Amit', 201),
('Temp', 202);

2. Add customer phone number in the Customer table
ALTER TABLE Customer
ADD phone_no VARCHAR(15);

3. Delete the phone number attribute from Customer
ALTER TABLE Customer
DROP COLUMN phone_no;

4. Find the names of all branches in the Loan relation
SELECT DISTINCT branch_name FROM Loan;

5. Find all customers who have a loan from the bank
SELECT B.cust_name, L.loan_no, L.amount
FROM Borrower B
JOIN Loan L
   ON B.loan_no = L.loan_no;

6. List all customers in alphabetical order who have a loan from Akurdi branch
SELECT DISTINCT B.cust_name
FROM Borrower B
JOIN Loan L
   ON B.loan_no = L.loan_no
WHERE L.branch_name = 'Akurdi'
```

```
ORDER BY B.cust_name ASC;
```

7. Find the average account balance at Chinchwad branch

```
SELECT AVG(balance) AS average_balance
FROM Account
WHERE branch_name = 'Chinchwad';
```

8. Find the number of depositors at each branch

```
SELECT A.branch_name,
      COUNT(DISTINCT D.cust_name) AS no_of_depositors
FROM Account A
JOIN Depositor D
   ON A.acc_no = D.acc_no
GROUP BY A.branch_name;
```

9. Find the total loan amount for each branch

```
SELECT branch_name,
      SUM(amount) AS total_loan_amount
FROM Loan
GROUP BY branch_name;
```

10. Find branches where the average loan amount is greater than ₹5,00,000

```
SELECT branch_name,
      AVG(amount) AS average_loan_amount
FROM Loan
GROUP BY branch_name
HAVING AVG(amount) > 500000;
```

PS 11

```
CREATE DATABASE BankDB;
USE BankDB;

CREATE TABLE Branch (
   branch_name VARCHAR(50) PRIMARY KEY,
   branch_city VARCHAR(50),
   assets DECIMAL(15,2) NOT NULL
);

CREATE TABLE Account (
   acc_no INT PRIMARY KEY,
   branch_name VARCHAR(50) NOT NULL,
   balance DECIMAL(12,2) NOT NULL,
   FOREIGN KEY (branch_name)
      REFERENCES Branch(branch_name)
);

CREATE TABLE Customer (
   cust_name VARCHAR(50) PRIMARY KEY,
   cust_street VARCHAR(100),
   cust_city VARCHAR(50)
);

CREATE TABLE Depositor (
   cust_name VARCHAR(50),
   acc_no INT,
   PRIMARY KEY (cust_name, acc_no),
   FOREIGN KEY (cust_name)
      REFERENCES Customer(cust_name),
   FOREIGN KEY (acc_no)
      REFERENCES Account(acc_no)
);

CREATE TABLE Loan (
   loan_no INT PRIMARY KEY,
   branch_name VARCHAR(50) NOT NULL,
   amount DECIMAL(12,2) NOT NULL,
   FOREIGN KEY (branch_name)
      REFERENCES Branch(branch_name)
);

CREATE TABLE Borrower (
   cust_name VARCHAR(50),
   loan_no INT,
   PRIMARY KEY (cust_name, loan_no),
   FOREIGN KEY (cust_name)
```

```
    REFERENCES Customer(cust_name),
  FOREIGN KEY (loan_no)
    REFERENCES Loan(loan_no)
);

INSERT INTO Branch VALUES
('Akurdi', 'Pune', 9000000),
('Chinchwad', 'Pune', 12000000),
('Nigdi', 'Mumbai', 8000000),
('Pimpri', NULL, 15000000);

INSERT INTO Account VALUES
(101, 'Akurdi', 15000),
(102, 'Akurdi', 9000),
(103, 'Chinchwad', 20000),
(104, 'Chinchwad', 10000),
(105, 'Nigdi', 14000);

INSERT INTO Customer VALUES
('Amit', 'MG Road', 'Pune'),
('Anita', NULL, 'Pune'),
('Rahul', 'Station Road', 'Mumbai'),
('Priya', 'FC Road', 'Pune'),
('Sneha', NULL, 'Mumbai');

INSERT INTO Depositor VALUES
('Amit', 101),
('Anita', 102),
('Rahul', 103),
('Priya', 104),
('Sneha', 105);

INSERT INTO Loan VALUES
(201, 'Akurdi', 15000),
(202, 'Akurdi', 13000),
(203, 'Chinchwad', 1400),
(204, 'Nigdi', 14000),
(205, 'Pimpri', 2000);

INSERT INTO Borrower VALUES
('Amit', 201),
('Anita', 202),
('Rahul', 203),
('Priya', 204),
('Sneha', 205);
```

1. Modify the assets attribute of the Branch table to Property.

```
ALTER TABLE Branch
```

CHANGE COLUMN assets Property DECIMAL(15,2) NOT NULL;

2. Find all loan numbers for loans made at Akurdi branch with a loan amount greater than 12,000.

```
SELECT loan_no
FROM Loan
WHERE branch_name = 'Akurdi'
AND amount > 12000;
```

3. Find the average account balance at each branch.

```
SELECT branch_name, AVG(balance) AS average_balance
FROM Account
GROUP BY branch_name;
```

4. Find the branches where the average account balance is greater than 12,000.

```
SELECT branch_name, AVG(balance) AS average_balance
FROM Account
GROUP BY branch_name
HAVING AVG(balance) > 12000;
```

5. Find the number of tuples in the Customer relation.

```
SELECT COUNT(*) AS total_customers
FROM Customer;
```

6. Calculate the total loan amount given by the bank.

```
SELECT SUM(amount) AS total_loan_amount
FROM Loan;
```

7. Delete all loans with a loan amount between 1,300 and 1,500.

```
DELETE FROM Loan
WHERE amount BETWEEN 1300 AND 1500;
```

or)

```
DELETE FROM Borrower
WHERE loan_no IN (
   SELECT loan_no
   FROM Loan
   WHERE amount BETWEEN 1300 AND 1500
);
```

```
DELETE FROM Loan
WHERE amount BETWEEN 1300 AND 1500;
```

8. Find the customers whose street address is not available.

```
SELECT cust_name
FROM Customer
WHERE cust_street IS NULL;
```

9. Find the branches where the branch city is available.

```
SELECT branch_name
```

```
FROM Branch
WHERE branch_city IS NOT NULL;
```

10. Find the names of customers whose names start with the letter 'A'.

```
SELECT cust_name
FROM Customer
WHERE cust_name LIKE 'A%';
```

PS 12

```
use ICEM
db.createCollection("employee")

db.employee.insertMany([
  {empid: 1, ename: "Amit", dept: "Computer", salary: 45000},
  {empid: 2, ename: "Priya", dept: "IT", salary: 50000}
])
```

1. Display the information of all employees.

```
db.employee.find()
```

2. Display the information of employees working in the Computer department.

```
db.employee.find({dept: "Computer"})
```

3. Display the information of employees working in the Computer or IT department using a logical operator.

```
db.employee.find({
  $or: [
    {dept: "Computer"},
    {dept: "IT"}
  ]
})
```

4. Display the information of employees whose salary is greater than ₹30,000.

```
db.employee.find({salary: {$gt: 30000}})
```

5. Display the information of employees whose salary is greater than or equal to ₹40,000 and who work in the Computer department.

```
db.employee.find({
  salary: {$gte: 40000},
  dept: "Computer"
})
```

6. Create an index on the ename field and drop the index created on the ename field in the employee collection.

```
db.employee.createIndex({ename: 1})
db.employee.getIndexes()
db.employee.dropIndex("ename_1")
```

7. Display the information of employees whose salary is between ₹20,000 and ₹40,000.

```
db.employee.find({
  salary: {$gte: 20000, $lte: 40000}
})
```

8. Display the information of employees whose department is not IT.

```
db.employee.find({dept: {$ne: "IT"}})
```

9. Display the total salary paid to employees in each department.

```
db.employee.aggregate([
  {
    $group: {
      _id: "$dept",
      total_salary: {$sum: "$salary"}
    }
  }
])
```

10. Display the average salary of employees in each department.

```
db.employee.aggregate([
  {
    $group: {
      _id: "$dept",
      average_salary: {$avg: "$salary"}
    }
  }
])
```

PS 13

```
Enterprise test> use Student
Enterprise Student> db.createCollection("student")
db.student.insertMany([
  {roll_no: 1, sname: "Amit", dept: "Computer", marks: 85},
  {roll_no: 2, sname: "Priya", dept: "IT", marks: 92},
  {roll_no: 3, sname: "Rahul", dept: "Computer", marks: 65}
])
```

1. Display the information of all students.

```
Enterprise test> db.student.find()
```

2. Display the information of students belonging to the Computer department.

```
Enterprise Student> db.student.find({dept: "Computer"})
```

3. Display the information of students belonging to the Computer or IT department using a logical operator.

```
db.student.find({
  $or: [
    {dept: "Computer"},
    {dept: "IT"}
  ]
})
```

4. Display the information of students whose marks are greater than 70.

```
db.student.find({marks: {$gt: 70}})
```

5. Display the information of students whose marks are greater than or equal to 80 and who belong to the Computer department.

```
db.student.find({
  marks: {$gte: 80},
  dept: "Computer"
})
```

6. Display the information of students whose marks are less than 40 or who belong to the IT department.

```
db.student.find({
  $or: [
    {marks: {$lt: 40}},
    {dept: "IT"}
  ]
})
```

7. Display the information of students whose marks are between 40 and 80.

```
db.student.find({
  marks: {$gte: 40, $lte: 80}
})
```

8. Display the information of students whose department is not IT.

```
db.student.find({dept: {$ne: "IT"}})
```

9. Display the total marks of students in each department.

```
db.student.aggregate([
  {
    $group: {
      _id: "$dept",
      total_marks: {$sum: "$marks"}
    }
  }
])
```

10. Create an index on the sname field and drop the index created on the sname field in the student collection.

```
db.student.createIndex({sname: 1})
db.student.getIndexes()
```

PS 14

1. Display all customers who live in Pune or Mumbai, arranged in alphabetical order of their names.
mysql> select * from Customer WHERE city = 'Pune' || city = 'Mumbai' ORDER BY cust_name ASC;

OR) select * from Customer WHERE city IN ('Pune', 'Mumbai') ORDER BY cust_name ASC;

2. Display the names of customers whose names contain the letter 'a'.
mysql> select * from Customer WHERE cust_name LIKE '%a%';

3. Display all customers whose phone number is NOT NULL.
mysql> select * from Customer WHERE phone IS NOT NULL;

4. Display customers who live in Pune and whose names start with 'P', or customers who live in Mumbai.
mysql> select * from Customer WHERE (city = 'Pune' AND cust_name LIKE 'P%') OR city = 'Mumbai' ;

5. Display the maximum loan amount and the average loan amount for each branch.
```
mysql> select b.branch_name,
    -> MAX(l.amount) AS max_loan,
    -> AVG(l.amount) AS avg_loan
    -> FROM Branch b
    -> JOIN Loan l ON b.branch_id = l.branch_id
    -> GROUP BY b.branch_id, b.branch_name;
```

6. Display branches whose total loan amount is greater than ₹1,00,000.
```
mysql> select b.branch_name, SUM(l.amount) AS total_amount
    -> FROM Branch b
    -> JOIN Loan l ON b.branch_id = l.branch_id
    -> GROUP BY b.branch_id, b.branch_name
```

-> HAVING SUM(l.amount) > 100000;

7. Display each account number along with its balance after adding ₹500. Do not modify the actual balance.
mysql> select acc_no, balance, balance+500 AS update_balance FROM Account;

8. Change the branch of account number 2005 to a different branch.
mysql> UPDATE Account SET branch_id = 105 WHERE acc_no = 2005;

9. Delete all customers whose phone number is NULL.
mysql> DELETE FROM Customer WHERE phone is NULL;

10. Display:a) Customers who have both an account and a loan using INTERSECT.
mysql> select cust_id, cust_name FROM Customer JOIN Account USING (cust_id)
-> INTERSECT
-> select cust_id, cust_name FROM Customer JOIN Loan USING (cust_id);

b) Customers who have an account but no loan using MINUS.
mysql> select cust_id, cust_name FROM Customer JOIN Account USING (cust_id)
-> MINUS
-> select cust_id, cust_name FROM Customer JOIN Loan USING (cust_id);